# Replication notebook

**Does Short Interest Predict Short-Horizon Returns? Evidence from S&P 500 Stocks**
BMF5324 AI and Statistics. Group: Lo Hao Yu, Michael Steven, Zhang Licheng, Vini Shrivastava.

This notebook rebuilds every number in the memo from the saved LSEG downloads, running top to bottom. Each printed figure carries the memo section it appears in, such as `[§5.2]`, and is also recorded as a pair (what the memo says, what this notebook computes). The last cell checks that every pair matches.

**How to run.** Keep this notebook in the folder with the files listed in the first code cell. Install the pinned packages with `pip install -r requirements.txt` (tested on Python 3.14.7), then choose *Run All*; it takes about two minutes and needs no LSEG access. The first code cell warns if any installed version differs from the pins.

**Seeds.** Nothing in the analysis is random. The only random numbers are drawn by the simulated self-test in `fm_regressions.py`, which fixes its seed (`numpy.random.default_rng(0)`), so every run gives identical results.

**Data and pull dates.**

| Data | Source | Pulled |
|---|---|---|
| S&P 500 membership, prices and volume, short interest, float, shares, market cap, analyst counts, book equity, corporate actions (`lseg_output_*`) | LSEG Workspace, through CodeBook (`lseg_pull.py`) | 21 October 2026 |
| Daily Fama-French 3 factors (`F-F_Research_Data_Factors_daily.csv`) | Kenneth French Data Library, CRSP 202608 vintage | 19 October 2026 |
| FINRA settlement and publication calendar (`finra_dates.csv`) | Built by `finra_dates.py` from FINRA's rule; checked against FINRA's published schedule and Nasdaq's dated releases | Schedule read 19 October 2026; Nasdaq dates checked 21 October 2026 |

**Not rerun here.** The LSEG download itself (`lseg_pull.py` without `--build`) runs only inside LSEG Workspace CodeBook and takes 2–4 hours. Its results are the saved `lseg_output_*` files, and step 3 rebuilds the panel from them with the same code. Figures quoted from the literature and the news (memo Sections 2 and 3) come from the cited sources and are not recomputed here. Section and table numbers follow the submitted memo.

| Step | Code | Output |
|---|---|---|
| 1. FINRA release calendar | `finra_dates.py` | `finra_dates.csv` |
| 2. Index membership | saved LSEG file | |
| 3. Panel from the saved LSEG files | `lseg_pull.py` (build mode) | `lseg_output_panel.csv` |
| 4. Cleaning | `clean_panel.py` | `panel_clean.csv` |
| 5. Fama-MacBeth regressions | `fm_regressions.py` | `regression_results.md`, `regression_coefs.csv` |
| 6. Memo Tables 2–6 and the numbers in the text | this notebook | |

In [1]:
```
import inspect
import platform
from pathlib import Path

import numpy as np
import pandas as pd
from IPython.display import Markdown, display

pd.set_option("display.width", 160)
print("Python", platform.python_version(), "| pandas", pd.__version__, "|
numpy", np.__version__)
from importlib.metadata import version
pins = dict(line.split("==") for line in
Path("requirements.txt").read_text().split() if "==" in line)
off = {pkg: (want, version(pkg)) for pkg, want in pins.items() if version(pkg)
!= want}
print("all package versions match requirements.txt" if not off else f"WARNING,
versions differ from the pins: {off}")

FILES = {
    "requirements.txt": "pinned package versions",
    "finra_dates.py": "FINRA settlement and publication calendar (writes
finra_dates.csv)",
    "lseg_pull.py": "LSEG download (CodeBook only) and the panel build",
    "clean_panel.py": "cleaning",
    "fm_regressions.py": "Fama-MacBeth regressions",
    "F-F_Research_Data_Factors_daily.csv": "Kenneth French daily 3 factors (for
idiosyncratic volatility)",
    "lseg_output_membership.csv": "S&P 500 members on each release date",
    "lseg_output_ric_map.csv": "index identifier -> listing used for data",
    "lseg_output_daily.csv.gz": "daily close and volume",
    "lseg_output_snapshots.csv": "short interest, float, shares, market cap,
analyst counts",
    "lseg_output_equity.csv": "quarterly common equity with announcement dates",
    "lseg_output_corporate_actions.csv": "split and spin-off adjustment
factors",
}
for f, what in FILES.items():
    assert Path(f).exists(), f"{f} is missing: run this notebook from the folder
that holds the replication files"
    print(f"{f:36s} {Path(f).stat().st_size / 1e6:7.2f} MB  {what}")
```

```
CHECKS = []   # (memo section, as the memo states it, as computed here)


def memo(where, text, *pairs):
    """Print a figure the memo uses, tagged with its section; each pair is (memo
text, computed text), checked at the end."""
    print(f"[{where}] {text}")
    CHECKS.extend((where, said, got) for said, got in pairs)
```

```
Python 3.14.7 | pandas 3.0.6 | numpy 2.5.3
all package versions match requirements.txt
requirements.txt                          0.00 MB  pinned package versions
finra_dates.py                            0.00 MB  FINRA settlement and publication
calendar (writes finra_dates.csv)
lseg_pull.py                              0.03 MB  LSEG download (CodeBook only) and
the panel build
clean_panel.py                            0.00 MB  cleaning
fm_regressions.py                         0.01 MB  Fama-MacBeth regressions
F-F_Research_Data_Factors_daily.csv       1.21 MB  Kenneth French daily 3 factors
(for idiosyncratic volatility)
lseg_output_membership.csv                4.44 MB  S&P 500 members on each release
date
lseg_output_ric_map.csv                   0.05 MB  index identifier -> listing used
for data
lseg_output_daily.csv.gz                 14.33 MB  daily close and volume
lseg_output_snapshots.csv                16.53 MB  short interest, float, shares,
market cap, analyst counts
lseg_output_equity.csv                    1.30 MB  quarterly common equity with
announcement dates
lseg_output_corporate_actions.csv         0.03 MB  split and spin-off adjustment
factors
```

# 1. FINRA release calendar (memo §4.1–§4.2)

FINRA settles short positions on the 15th and the last business day of each month and publishes them seven NYSE trading days later. `finra_dates.py` rebuilds that schedule, checks it against FINRA's 28 published 2025–26 rows and six dated Nasdaq releases, and stops with an error if any date disagrees.

In [2]:
```
# %run shows a failed self-check but keeps going, so delete the old output: a
failure then stops at the next read
Path("finra_dates.csv").unlink(missing_ok=True)
%run finra_dates.py
dates =pd.read_csv("finra_dates.csv", parse_dates=["Settlement", "Publication"])
lag = (dates["Publication"] - dates["Settlement"]).dt.days
full_years = dates.groupby(dates["Settlement"].dt.year).size().loc[2017:2025]
first, last = dates["Settlement"].iloc[0], dates["Settlement"].iloc[-1]
memo("§4.1", f"{len(dates)} releases, settlement dates {first:%d %B %Y} to
{last:%d %B %Y}, {full_years.min()} per full year",
     ("240", str(len(dates))), ("15 September 2016", f"{first:%d %B %Y}"), ("31
August 2026", f"{last:%d %B %Y}"),
     ("24", str(full_years.min())))
memo("§4.2", f"lag from settlement to publication: {lag.min()}–{lag.max()}
calendar days",
     ("9–13", f"{lag.min()}–{lag.max()}"))
first_pub, last_pub = dates["Publication"].iloc[0],
dates["Publication"].iloc[-1]
```

```
memo("§2.2", f"publication dates {first_pub:%d %B %Y} to {last_pub:%d %B %Y}",
     ("26 September 2016", f"{first_pub:%d %B %Y}"), ("10 September 2026", f"
{last_pub:%d %B %Y}"))
```

```
240 dates, 2016-09-15 to 2026-08-31; matches FINRA's 28 published rows and 6
Nasdaq release dates -> finra_dates.csv
[§4.1] 240 releases, settlement dates 15 September 2016 to 31 August 2026, 24 per
full year
[§4.2] lag from settlement to publication: 9–13 calendar days
[§2.2] publication dates 26 September 2016 to 10 September 2026
```

## 2. Index membership (memo §4.1)

S&P 500 constituents downloaded from LSEG on each publication date, so firms that join or leave the index stay in the sample for the dates they were members.

In [3]:
```
mem = pd.read_csv("lseg_output_membership.csv", parse_dates=["Date"])
per_date = mem.groupby("Date").size()
memo("§4.1", f"constituent lists: {per_date.min()}-{per_date.max()} securities
per release, "
             f"{mem['RIC'].nunique()} distinct identifiers over
{mem['Date'].nunique()} releases",
     ("503-507", f"{per_date.min()}-{per_date.max()}"), ("695",
str(mem["RIC"].nunique())))
```

```
[§4.1] constituent lists: 503-507 securities per release, 695 distinct identifiers
over 240 releases
```

## 3. Panel built from the saved LSEG files (memo §4.1, §4.3, Table 1)

`lseg_pull.main(..., build_only=True)` is the same code that ran in CodeBook, reading the saved downloads instead of calling LSEG. It maps each index identifier to one tradable listing, puts short interest, float and volume into each date's share units with LSEG's split and spin-off factors (separately for each share class), uses book equity only after its announcement date (quarter end + 90 days when there is none), and builds every return window from daily closes. Self-checks stop the build if the settlement-date short interest is not the published figure, if Alphabet's two classes disagree, or if any stock's turnover is implausibly low.

In [4]:
```
import lseg_pull as L

L.main(False, build_only=True)
```

```
240 dates | 691 stocks | 120,970 panel rows | consolidated volume for 685/695 |
failed requests: 0
Share of panel rows with a value:
Publication             1.000
Settlement              1.000
RIC                     1.000
DataRIC                 1.000
Name                    1.000
SI_float                0.922
SI_shares               0.922
dSI_float               0.917
MktCap                  1.000
LogMV                   1.000
Analysts                0.986
AnalystCov              0.986
BookEquity              0.999
BookPeriodEnd           0.999
BM                      0.999
FwdRet5                 0.999
FwdRet5_MktAdj          0.999
FwdRet5_MktAdj_skip1    0.999
FwdRet2w                0.995
FwdRet2w_MktAdj         0.995
FwdRet2w_t2_MktAdj      0.990
LagRet1m                0.999
LagRet6m                0.998
PctZero                 1.000
IdioVOL                 0.999
Turnover                1.000
SI_x_Analysts           0.913
SI_x_LogMV              0.922
Saved lseg_output_*.csv
```

In [5]:
```
# Table 1's windows: forward returns over HOLD = 5 trading days from the
release-day close; past month = 21 days,
# months -6 to -2 = days 126 to 21 back, turnover and zero-return days over 21
days, idiosyncratic volatility over 63
print(inspect.getsource(L.window_vars))
```

```
def window_vars(obs, dly, ff):
    """Return-window variables. k = the release day; backward windows end at its
close, forward windows start there.
    FwdRet5 = the 5 trading days after the release. Bi-weekly returns run from one
release's close to the next
    release's close (t+1) and on to the one after (t+2), so consecutive windows
tile time like FINRA's releases."""
    px = dly.pivot(index="Date", columns="RIC", values="Close").sort_index()
    # trading days = days most stocks have a close; LSEG's .SPX repeats its last
value on some holidays
    # (10 of them in Sep 2021 - Sep 2022), which would otherwise blank every
window that touches them
    n_px = px.drop(columns=INDEX).notna().sum(axis=1)
    px = px[n_px >= n_px.median() / 2]
    p, v =px.to_numpy(float), dly.pivot(index="Date", columns="RIC",
values="Volume").reindex(px.index).to_numpy(float)
    ret = p / np.vstack([np.full((1, p.shape[1]), np.nan), p[:-1]]) - 1
    col = {r: i for i, r in enumerate(px.columns)}
    m, n = col[INDEX], len(px)
    if ff is not None:
        f = ff.reindex(px.index)
        X = np.column_stack([np.ones(n), f[["MktRF", "SMB", "HML"]].to_numpy()])
        rf = f["RF"].to_numpy()
    pubs = pd.DatetimeIndex(sorted(obs["Publication"].unique()))
    missing = pubs[~pubs.isin(px.index)]
    assert missing.empty, f"release days with no trading: {list(missing.date)};
forward returns would start too early"
    kp = list(px.index.searchsorted(pubs, side="right") - 1) + [None, None]   #
shortcut: no t+1 for the last date
    ret_mkt = lambda j, a, b: (p[b, j] / p[a, j] - 1, p[b, j] / p[a, j] - p[b, m]
/ p[a, m])
    out = []
    for ric, i, k in zip(obs["DataRIC"], pubs.get_indexer(obs["Publication"]),
                         px.index.searchsorted(obs["Publication"], side="right") -
1):
        j, r = col.get(ric), {}
        if j is not None and k >= 126:
            if k + HOLD < n:
                r["FwdRet5"], r["FwdRet5_MktAdj"] = ret_mkt(j, k, k + HOLD)
            if k + 1 + HOLD < n:
                r["FwdRet5_MktAdj_skip1"] = ret_mkt(j, k + 1, k + 1 + HOLD)[1]
            k1, k2 = kp[i + 1], kp[i + 2]
            if k1 is not None:
                r["FwdRet2w"], r["FwdRet2w_MktAdj"] = ret_mkt(j, k, k1)
                if k2 is not None:
                    r["FwdRet2w_t2_MktAdj"] = ret_mkt(j, k1, k2)[1]
            r["LagRet1m"] = p[k, j] / p[k - 21, j] - 1
            r["LagRet6m"] = p[k - 21, j] / p[k - 126, j] - 1
            v21, r21 = v[k - 20:k + 1, j], ret[k - 20:k + 1, j]
            v21, r21 = v21[~np.isnan(v21)], r21[~np.isnan(r21)]
            r["AvgVolume21"] = v21.mean() if len(v21) else np.nan
            r["PctZero"] = (r21 == 0).mean() if len(r21) else np.nan
            if ff is not None:
                y, x = ret[k - 62:k + 1, j] - rf[k - 62:k + 1], X[k - 62:k + 1]
                ok = ~np.isnan(y) & ~np.isnan(x).any(axis=1)
                if ok.sum() >= 50:   # shortcut: needs 50 of 63 days; French data
lags 1-2 months, so the last dates may miss
                    e = y[ok] - x[ok] @ np.linalg.lstsq(x[ok], y[ok], rcond=None)
[0]
```

```
                    r["IdioVOL"] = e.std(ddof=4)
        out.append(r)
    return pd.DataFrame(out, index=obs.index, columns=WINDOW_COLS)
```

In [6]:
```
raw = pd.read_csv("lseg_output_panel.csv", parse_dates=["Publication"])
n_dates = raw["Publication"].nunique()
releases_per_firm = raw.groupby("DataRIC")["Publication"].nunique()
always, partial = int((releases_per_firm == n_dates).sum()),
int((releases_per_firm < n_dates).sum())
memo("§4.1", f"built panel: {len(raw):,} stock-release observations on
{raw['DataRIC'].nunique()} securities; "
             f"{always} are members at all {n_dates} releases and {partial}
enter or leave",
     ("120,970", f"{len(raw):,}"), ("691", str(raw["DataRIC"].nunique())),
("353", str(always)), ("338", str(partial)))

no_si = raw.groupby("DataRIC")["SI_float"].transform(lambda s: s.isna().all())
# LSEG keeps no SI history for these
delisted = raw.loc[no_si, "RIC"].str.contains("^", regex=False).mean()
# "^" marks a delisted identifier
by_year = 100 * no_si.groupby(raw["Publication"].dt.year).mean()
early, late = by_year.loc[:2020], by_year.loc[2021:]
memo("§4.1, §6.2", f"{raw.loc[no_si, 'DataRIC'].nunique()} securities have no
short-interest history "
                   f"({delisted:.0%} of them delisted): {no_si.mean():.1%} of
observations",
     ("101", str(raw.loc[no_si, "DataRIC"].nunique())), ("7.7%", f"
{no_si.mean():.1%}"))
memo("§5.5, §6.2", f"their share of observations a year in 2016–2020:
{early.min():.0f}–{early.max():.0f}%",
     ("9–17%", f"{early.min():.0f}–{early.max():.0f}%"))
memo("§5.5", f"their share of observations a year in 2021–2026:
{late.min():.0f}–{late.max():.0f}%",
     ("1–8%", f"{late.min():.0f}–{late.max():.0f}%"))
by_year.round(1).rename("% of observations without SI history").to_frame().T
```

```
[§4.1] built panel: 120,970 stock-release observations on 691 securities; 353 are
members at all 240 releases and 338 enter or leave
[§4.1, §6.2] 101 securities have no short-interest history (100% of them
delisted): 7.7% of observations
[§5.5, §6.2] their share of observations a year in 2016–2020: 9–17%
[§5.5] their share of observations a year in 2021–2026: 1–8%
```

Out[6]:

| Publication | 2016 | 2017 | 2018 | 2019 | 2020 | 2021 | 2022 | 2023 | 2024 | 2025 | 2026 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **% of observations without SI history** | 16.7 | 15.1 | 13.1 | 10.7 | 8.7 | 7.5 | 6.0 | 4.4 | 3.4 | 2.1 | 0.7 |

**What this shows.** The panel follows the index as it stood on each date, so firms that later left the S&P 500 stay in for the dates they belonged to it. The price is the delisted firms for which LSEG keeps no short interest. They sit mostly in the early years, so the 2016–2020 sample is missing more of its members than the later one, which matters when the two periods are compared in Memo Table 5.

# 4. Cleaning (memo §4.1, §4.3)

`clean_panel.py` drops Viatris before its November 2020 merger, sets book-to-market to missing when book equity is not positive, keeps one share class per company and date, and winsorizes the continuous variables at 1%/99% within each release.

In [7]:
```
Path("panel_clean.csv").unlink(missing_ok=True)   # see step 1: a failed run must not leave the last file behind
%run clean_panel.py
clean =pd.read_csv("panel_clean.csv", usecols=["Publication", "DataRIC"])
memo("§4.1", f"after cleaning: {len(clean):,} observations on {clean['DataRIC'].nunique()} securities",
     ("119,879", f"{len(clean):,}"), ("688", str(clean["DataRIC"].nunique())))
memo("§4.3", f"book-to-market set to missing for {neg_bm:,} observations with book equity <= 0 (neg_bm, from clean_panel.py)",
     ("6,629", f"{neg_bm:,}"))
```

```
120,970 rows -> 119,879: dropped 100 pre-merger Viatris rows and 991 second share-class rows; set BM missing in 6,629 rows with BM <= 0; winsorized 15 variables at 1%/99% per date -> panel_clean.csv
[§4.1] after cleaning: 119,879 observations on 688 securities
[§4.3] book-to-market set to missing for 6,629 observations with book equity <= 0 (neg_bm, from clean_panel.py)
```

# 5. Fama-MacBeth regressions (memo §4.4–§4.5)

One OLS cross-section per release, with every regressor standardized within the release (coefficients are basis points of five-day return per one standard deviation; interactions are products of the standardized variables). The reported coefficient is the mean of the 240 slopes, with a Newey-West t-statistic (4 lags). The estimator:

In [8]:
```
import fm_regressions as F

print(inspect.getsource(F.nw_tstat))
print(inspect.getsource(F.fm))
WORDS = {4: "four", 7: "seven"}
memo("§4.2", f"Newey-West lags: {F.LAGS}", ("4 lags", f"{F.LAGS} lags"))
memo("§4.4", "the same, in words", ("four lags", f"{WORDS.get(F.LAGS, F.LAGS)} lags"))
memo("§1, §4.3, §4.4", f"controls: {', '.join(F.CTRL)}", ("seven",
WORDS.get(len(F.CTRL), str(len(F.CTRL)))))
print(f"each cross-section needs at least {F.MIN_N} stocks (a pipeline setting the memo does not quote)")
for f in ["regression_results.md", "regression_coefs.csv"]:   # see step 1; the next cell must start with %%capture
    Path(f).unlink(missing_ok=True)
```

```
def nw_tstat(b, lags=LAGS):
    """t-statistic of the mean of a coefficient series, Newey-West (Bartlett)
standard error."""
    e, T = b - b.mean(), len(b)
    s = e @ e / T + 2 * sum((1 - l / (lags + 1)) * (e[l:] @ e[:-l]) / T for l in
range(1, lags + 1))
    return b.mean() / np.sqrt(s / T)

def fm(df, y, terms, z=True, weight=None):
    """Fama-MacBeth: one cross-sectional regression per date, OLS or WLS with
weight (a column, e.g. MktCap).
    terms may hold interactions 'a:b'."""
    base = sorted({t for term in terms for t in term.split(":")})
    d = df[["Publication", y] + base + ([weight] if weight else [])].dropna()
    rows = []
    for date, g in d.groupby("Publication"):
        if len(g) < MIN_N:
            continue
        x = g[base].astype(float)
        if z:
            zc = [c for c in base if c not in NO_Z]
            x[zc] = (x[zc] - x[zc].mean()) / x[zc].std()
        X = np.column_stack([np.ones(len(g))] + [np.prod([x[t] for t in
term.split(":")], axis=0) for term in terms])
        yy = g[y].to_numpy(float) * 1e4                          # basis points
        sw = np.sqrt(g[weight].to_numpy(float)) if weight else np.ones(len(g))
        b, *_ = np.linalg.lstsq(X * sw[:, None], yy * sw, rcond=None)
        resid, dev = (yy - X @ b) * sw, (yy - np.average(yy, weights=sw ** 2)) *
sw
        r2 = 1 - resid @ resid / (dev @ dev)
        k = X.shape[1] - 1
        rows.append([date, len(g), 1 - (1 - r2) * (len(g) - 1) / (len(g) - k - 1),
*b])
    B = pd.DataFrame(rows, columns=["date", "n", "adjR2", "const"] + terms)
    coefs = {t: (B[t].mean(), nw_tstat(B[t].to_numpy())) for t in ["const"] +
terms}
    return {"coefs": coefs, "dates": len(B), "avg_n": B["n"].mean(), "adjR2":
B["adjR2"].mean(), "series": B}

[§4.2] Newey-West lags: 4
[§4.4] the same, in words
[§1, §4.3, §4.4] controls: LogMV, BM, LagRet1m, LagRet6m, Turnover, IdioVOL,
PctZero
each cross-section needs at least 30 stocks (a pipeline setting the memo does not
quote)
```

`fm_regressions.py` runs a self-test on simulated data, then every model in the memo, and writes all results to `regression_results.md` (shown below) and every coefficient to `regression_coefs.csv`. Its Table 7 holds memo §6.2's comparison of delisted firms without short interest against the firms in the regressions; its Table 8 holds the controls-only check. These are the pipeline's own table numbers (1–8), not the memo's; the memo's tables follow in section 6 of this notebook.

In [9]:
```
%%capture fm_log
%run fm_regressions.py
```

In [10]:

```
display(Markdown(Path("regression_results.md").read_text(encoding="utf-8")))
C = pd.read_csv("regression_coefs.csv").drop_duplicates(["model",
"term"]).set_index(["model", "term"])
models = C.index.get_level_values("model").unique()
memo("§6.2", f"{len(models)} regression models: the four main ones and
{len(models) - 4} variants",
     ("about twenty", "about twenty" if 18 <= len(models) - 4 <= 22 else
str(len(models) - 4)))


def coef(model, term="SI_float"):
    return C.loc[(model, term), "coef_bps"], C.loc[(model, term), "t_NW"]


def cell(model, term="SI_float"):
    """Memo format: coefficient with stars, t-statistic in brackets."""
    if (model, term) not in C.index:
        return ""
    b, t = coef(model, term)
    return f"{b:.2f}{F.stars(t)} ({t:.2f})"
```

# Fama-MacBeth results

Panel: panel_clean.csv (119,879 rows, 240 FINRA dates, 688 stocks). Dependent variable unless noted: FwdRet5_MktAdj (the proposal's main specification).

## Table 1. H1 and H2: next 5-trading-day market-adjusted return (bps per 1 SD), Fama-MacBeth

| | (1) SI only | (2) H1 | (3) H2: SI × coverage | (4) H2 robustness: SI × size |
|---|---|---|---|---|
| SI / float | -3.99 (-1.38) | -6.22 (-2.89)*** | -5.04 (-2.36)** | -0.89 (-0.34) |
| log market cap | | 3.48 (1.38) | 2.88 (1.15) | 6.75 (2.20)** |
| book-to-market | | -2.39 (-0.70) | -2.34 (-0.69) | -2.08 (-0.62) |
| return, past month | | -3.30 (-0.80) | -3.24 (-0.78) | -3.91 (-0.97) |
| return, months -6 to -2 | | -1.88 (-0.42) | -1.71 (-0.39) | -2.43 (-0.54) |
| turnover | | 4.13 (1.20) | 3.24 (0.95) | 4.11 (1.18) |
| idiosyncratic volatility | | 1.87 (0.63) | 1.54 (0.53) | 1.75 (0.59) |
| % zero-return days | | 0.27 (0.27) | 0.40 (0.39) | 0.40 (0.39) |
| analyst coverage, ln(1+N) | | | 2.31 (1.01) | |
| SI × analyst coverage | | | 1.75 (1.22) | |
| SI × size | | | | 5.95 (2.94)*** |
| intercept | -7.52 (-1.28) | -7.93 (-1.34) | -7.55 (-1.29) | -5.05 (-0.89) |
| dates | 240 | 240 | 240 | 240 |
| avg. stocks per date | 461 | 433 | 429 | 433 |
| avg. adj. $R^2$ | 0.018 | 0.135 | 0.142 | 0.137 |

t-statistics (Newey-West, 4 lags) in brackets; * 10%, ** 5%, *** 1%. H1 predicts a positive SI coefficient; H2 predicts a negative SI × coverage (and SI × size) coefficient. The 5-day return runs from the close of the release day to the close 5 trading days later.

## Table 2. H2 check: model (2) within analyst-coverage terciles

| | (2) low coverage (bottom third) | (2) high coverage (top third) |
|---|---|---|
| SI / float | -8.83 (-2.80)*** | -3.71 (-0.99) |
| log market cap | 0.38 (0.16) | 6.43 (1.63) |

| | (2) low coverage (bottom third) | (2) high coverage (top third) |
|---|---|---|
| book-to-market | -1.39 (-0.42) | -3.62 (-0.87) |
| return, past month | -3.29 (-0.81) | -3.55 (-0.79) |
| return, months -6 to -2 | -0.96 (-0.21) | -1.46 (-0.26) |
| turnover | 2.12 (0.52) | 4.22 (0.91) |
| idiosyncratic volatility | 1.62 (0.46) | 6.06 (1.50) |
| % zero-return days | 1.29 (0.71) | 0.96 (0.49) |
| intercept | -11.51 (-1.58) | -3.11 (-0.62) |
| dates | 240 | 240 |
| avg. stocks per date | 144 | 140 |
| avg. adj. $R^2$ | 0.135 | 0.152 |

The proposal says H2 fails if the SI coefficient is not positive among low-coverage firms. Terciles are formed within each date from ln(1 + analysts).

## Table 3. Robustness (value-weighted = WLS by market cap, as the proposal specifies)

| | (2) baseline | (2) value-weighted | (3) value-weighted | (2) raw panel (no cleaning) | (2) keep BE ≤ 0 (dummy) | (2) SI / shares out | (2) skip a day |
|---|---|---|---|---|---|---|---|
| SI / float | -6.22 (-2.89)*** | -4.80 (-1.35) | -3.27 (-0.95) | -11.02 (-1.72)* | -5.94 (-2.82)*** | | -4.32 (-1.70)* |
| log market cap | 3.48 (1.38) | 4.80 (1.09) | 4.84 (1.24) | 4.11 (1.56) | 3.12 (1.27) | 3.39 (1.34) | 2.39 (0.83) |
| book-to-market | -2.39 (-0.70) | -3.82 (-0.81) | -4.00 (-0.87) | -7.36 (-1.44) | | -2.40 (-0.71) | -2.57 (-0.73) |
| return, past month | -3.30 (-0.80) | -6.83 (-1.39) | -7.38 (-1.53) | -2.62 (-0.59) | -3.04 (-0.75) | -3.34 (-0.81) | -1.04 (-0.17) |
| return, months -6 to -2 | -1.88 (-0.42) | -4.42 (-0.80) | -3.20 (-0.59) | -0.08 (-0.02) | -1.65 (-0.37) | -1.84 (-0.41) | -5.27 (-1.17) |
| turnover | 4.13 (1.20) | 9.99 (1.98)** | 9.22 (1.87)* | 20.26 (2.59)*** | 3.73 (1.05) | 4.56 (1.29) | 1.29 (0.33) |
| idiosyncratic volatility | 1.87 (0.63) | -1.12 (-0.22) | -1.60 (-0.31) | 0.81 (0.24) | 1.91 (0.66) | 1.71 (0.58) | 0.95 (0.34) |
| % zero-return days | 0.27 (0.27) | 2.84 (1.22) | 3.08 (1.34) | 0.01 (0.01) | 0.36 (0.38) | 0.27 (0.26) | 1.05 (0.97) |
| analyst coverage, ln(1+N) | | | 3.04 (1.19) | | | | |

| | (2) baseline | (2) value-weighted | (3) value-weighted | (2) raw panel (no cleaning) | (2) keep BE ≤ 0 (dummy) | (2) SI / shares out | (2) skip a day |
|---|---|---|---|---|---|---|---|
| SI × analyst coverage | | | 3.79 (1.34) | | | | |
| book-to-market (0 if BE ≤ 0) | | | | | -2.38 (-0.69) | | |
| negative book equity (dummy) | | | | | 0.23 (0.04) | | |
| SI / shares out | | | | | | -6.56 (-2.80)*** | |
| intercept | -7.93 (-1.34) | -6.77 (-0.98) | -6.52 (-0.96) | -7.32 (-1.25) | -7.60 (-1.33) | -7.93 (-1.34) | -6.42 (-0.91) |
| dates | 240 | 240 | 240 | 240 | 240 | 240 | 240 |
| avg. stocks per date | 433 | 433 | 429 | 464 | 460 | 433 | 433 |
| avg. adj. $R^2$ | 0.135 | 0.227 | 0.241 | 0.127 | 0.133 | 0.135 | 0.133 |

## Table 4. Bi-weekly horizons (professor): return from one release to the next (t+1) and the one after (t+2)

| | (2) 5 days (main) | (2) 2 weeks, t+1 | (2) 2 weeks, t+2 | (3) 2 weeks, t+1 |
|---|---|---|---|---|
| SI / float | -6.22 (-2.89)*** | -10.83 (-3.34)*** | -11.09 (-3.47)*** | -9.48 (-2.96)*** |
| log market cap | 3.48 (1.38) | 4.32 (1.12) | 3.50 (0.97) | 2.20 (0.53) |
| book-to-market | -2.39 (-0.70) | -1.66 (-0.29) | -1.85 (-0.33) | -1.76 (-0.30) |
| return, past month | -3.30 (-0.80) | -7.28 (-1.27) | 0.83 (0.16) | -7.37 (-1.30) |
| return, months -6 to -2 | -1.88 (-0.42) | 3.51 (0.66) | 2.54 (0.50) | 3.76 (0.71) |
| turnover | 4.13 (1.20) | 10.69 (2.09)** | 11.03 (2.29)** | 9.30 (1.85)* |
| idiosyncratic volatility | 1.87 (0.63) | 0.76 (0.17) | 1.11 (0.24) | 0.29 (0.06) |
| % zero-return days | 0.27 (0.27) | -0.25 (-0.16) | -1.41 (-0.86) | 0.09 (0.06) |
| analyst coverage, ln(1+N) | | | | 5.35 (1.44) |
| SI × analyst coverage | | | | 1.22 (0.62) |
| intercept | -7.93 (-1.34) | -11.80 (-1.48) | -11.64 (-1.45) | -11.45 (-1.44) |
| dates | 240 | 239 | 238 | 239 |
| avg. stocks per date | 433 | 433 | 433 | 429 |

| | (2) 5 days (main) | (2) 2 weeks, t+1 | (2) 2 weeks, t+2 | (3) 2 weeks, t+1 |
|---|---|---|---|---|
| avg. adj. $R^2$ | 0.135 | 0.132 | 0.132 | 0.139 |

The bi-weekly return runs from the close of release t to the close of release t+1, so consecutive returns tile time exactly like FINRA's twice-monthly releases.

## Table 5. Short covering (professor): the 2-week % change in short interest

| | 5 days: + ΔSI | 5 days: SI × ΔSI | 2 weeks: + ΔSI | 2 weeks: SI × ΔSI |
|---|---|---|---|---|
| SI / float | -6.30 (-2.94)*** | -6.77 (-3.12)*** | -11.17 (-3.43)*** | -11.30 (-3.36)*** |
| ΔSI (2-week % change) | -2.60 (-1.78)* | -2.64 (-1.75)* | -3.10 (-1.53) | -3.29 (-1.58) |
| log market cap | 3.42 (1.34) | 3.16 (1.25) | 4.02 (1.03) | 3.82 (0.98) |
| book-to-market | -2.38 (-0.71) | -2.38 (-0.70) | -2.00 (-0.35) | -2.07 (-0.36) |
| return, past month | -2.93 (-0.71) | -2.62 (-0.64) | -6.52 (-1.14) | -6.17 (-1.08) |
| return, months -6 to -2 | -2.20 (-0.49) | -2.13 (-0.48) | 3.24 (0.62) | 3.04 (0.58) |
| turnover | 4.25 (1.22) | 3.82 (1.08) | 10.64 (2.12)** | 10.48 (2.08)** |
| idiosyncratic volatility | 1.65 (0.55) | 1.83 (0.60) | 0.52 (0.11) | 0.59 (0.13) |
| % zero-return days | 0.41 (0.40) | 0.37 (0.36) | -0.04 (-0.02) | -0.16 (-0.10) |
| SI × ΔSI | | 2.29 (1.37) | | 2.06 (1.02) |
| intercept | -7.88 (-1.33) | -8.09 (-1.37) | -11.67 (-1.46) | -11.79 (-1.48) |
| dates | 239 | 239 | 238 | 238 |
| avg. stocks per date | 433 | 433 | 432 | 432 |
| avg. adj. $R^2$ | 0.136 | 0.137 | 0.133 | 0.133 |

A falling SI (negative ΔSI) is evidence of short covering, which the professor expected to push prices up, i.e. a negative ΔSI coefficient.

## Table 6. Subperiods (delisted-firm SI gap: 9-17% of rows a year in 2016-2020, 1-8% in 2021-2026)

| | (2) 2016-2020 | (3) 2016-2020 | (2) 2021-2026 | (3) 2021-2026 |
|---|---|---|---|---|
| SI / float | -10.64 (-3.05)*** | -9.43 (-2.90)*** | -2.89 (-1.16) | -1.75 (-0.66) |
| log market cap | 1.78 (0.43) | 1.94 (0.53) | 4.76 (1.54) | 3.59 (1.07) |
| book-to-market | -13.34 (-2.80)*** | -13.13 (-2.74)*** | 5.84 (1.42) | 5.77 (1.40) |
| return, past month | -5.73 (-0.90) | -5.85 (-0.89) | -1.47 (-0.28) | -1.28 (-0.25) |
| return, months -6 to -2 | 2.98 (0.55) | 2.82 (0.53) | -5.53 (-0.83) | -5.12 (-0.79) |
| turnover | 5.13 (1.19) | 4.13 (0.99) | 3.38 (0.66) | 2.57 (0.51) |

| | (2) 2016-2020 | (3) 2016-2020 | (2) 2021-2026 | (3) 2021-2026 |
|---|---|---|---|---|
| idiosyncratic volatility | -1.63 (-0.50) | -1.91 (-0.59) | 4.50 (1.01) | 4.13 (0.94) |
| % zero-return days | -0.46 (-0.27) | -0.23 (-0.13) | 0.82 (0.67) | 0.87 (0.70) |
| analyst coverage, ln(1+N) | | 1.25 (0.36) | | 3.10 (1.03) |
| SI × analyst coverage | | 3.24 (1.54) | | 0.63 (0.33) |
| intercept | -12.73 (-1.82)* | -12.19 (-1.77)* | -4.32 (-0.49) | -4.07 (-0.46) |
| dates | 103 | 103 | 137 | 137 |
| avg. stocks per date | 415 | 411 | 446 | 443 |
| avg. adj. $R^2$ | 0.119 | 0.128 | 0.147 | 0.153 |

## Table 7. Delisted firms without SI vs firms in the regressions (per-date means, averaged)

| variable | in regressions | delisted, no SI | difference (t) |
|---|---|---|---|
| next 5-day adj. return (bps) | -7.5202 | -11.4809 | -3.9607 (-0.62) |
| log market cap | 24.2523 | 23.6143 | -0.6380 (-28.22)*** |
| book-to-market | 0.3684 | 0.4342 | 0.0658 (7.90)*** |
| return, past month | 0.0101 | 0.0086 | -0.0015 (-0.81) |
| return, months -6 to -2 | 0.0497 | 0.0415 | -0.0082 (-0.98) |
| turnover | 0.0085 | 0.0107 | 0.0022 (11.88)*** |
| idiosyncratic volatility | 0.0146 | 0.0160 | 0.0014 (3.54)*** |

Difference t-statistics are Newey-West over dates.

## Table 8. Controls-only model with and without the firms lacking SI

| | all firms | firms with SI only |
|---|---|---|
| log market cap | 5.42 (2.25)** | 5.45 (2.21)** |
| book-to-market | -1.50 (-0.44) | -1.95 (-0.58) |
| return, past month | -3.53 (-0.82) | -3.85 (-0.92) |
| return, months -6 to -2 | -1.81 (-0.40) | -1.75 (-0.39) |
| turnover | 1.73 (0.54) | 1.66 (0.51) |
| idiosyncratic volatility | 1.17 (0.42) | 1.30 (0.44) |
| % zero-return days | 0.07 (0.07) | 0.32 (0.31) |
| intercept | -8.57 (-1.44) | -7.93 (-1.34) |
| dates | 240 | 240 |

| | all firms | firms with SI only |
|---|---|---|
| avg. stocks per date | 470 | 433 |
| avg. adj. $R^2$ | 0.128 | 0.130 |

If the coefficients barely move, leaving out the delisted firms does not distort the cross-section.

```
[§6.2] 25 regression models: the four main ones and 21 variants
```

# 6. Memo exhibits

Memo Table 1 (variable definitions) and Exhibits 1 and 2 (rival mechanisms, design choices) hold no estimates. Tables 2–6 follow in the memo's order; Table 6 is in the memo's Appendix.

## Memo Table 2. Descriptive statistics, H1 regression sample (memo §5.1)

In [11]:
```
p = F.load("panel_clean.csv")
s = p.dropna(subset=[F.Y] + F.H1)          # the H1 regression sample
desc = pd.DataFrame({
    "Short interest / float (%)": 100 * s["SI_float"], "5-day market-adjusted
return (%)": 100 * s[F.Y],
    "Market cap ($ billion)": s["MktCap"] / 1e9, "Book-to-market": s["BM"],
    "Turnover (% of shares per day)": 100 * s["Turnover"], "Zero-return days (%
of last 21 days)": 100 * s["PctZero"],
    "Idiosyncratic volatility (% per day)": 100 * s["IdioVOL"], "Analysts
(number)": s["Analysts"],
    "Return, past month (%)": 100 * s["LagRet1m"], "Return, months -6 to -2
(%)": 100 * s["LagRet6m"]})
table2 = desc.describe(percentiles=[0.1, 0.5, 0.9]).T[["mean", "std", "10%",
"50%", "90%"]]
table2.columns = ["Mean", "SD", "P10", "Median", "P90"]


def memo_format(row, v):   # the memo shows market cap and analyst counts with
fewer decimals
    if row == "Market cap ($ billion)":
        return [f"{x:.1f}" for x in v]
    if row == "Analysts (number)":
        return [f"{v.iloc[0]:.1f}", f"{v.iloc[1]:.1f}"] + [f"{x:.0f}" for x in
v.iloc[2:]]
    return [f"{x:.2f}" for x in v]


table2_memo = pd.DataFrame([memo_format(r, table2.loc[r]) for r in
table2.index], index=table2.index, columns=table2.columns)
MEMO_TABLE2 = {   # as printed in the memo
    "Short interest / float (%)": "2.95 2.72 0.93 2.04 6.09", "5-day market-
adjusted return (%)": "-0.08 3.67 -4.20 -0.07 4.04",
    "Market cap ($ billion)": "80.3 236.3 10.3 29.0 155.8", "Book-to-market":
"0.37 0.31 0.07 0.29 0.76",
    "Turnover (% of shares per day)": "0.85 0.59 0.39 0.68 1.45", "Idiosyncratic
volatility (% per day)": "1.46 0.66 0.83 1.30 2.29",
```

```
    "Analysts (number)": "18.3 7.8 9 18 28", "Return, past month (%)": "1.00
8.69 -8.92 1.04 10.92",
    "Zero-return days (% of last 21 days)": "0.44 1.50 0.00 0.00 0.00",
    "Return, months -6 to -2 (%)": "4.93 19.61 -17.52 3.96 27.56"}
CHECKS.extend(("Table 2", said, " ".join(table2_memo.loc[row])) for row, said in
MEMO_TABLE2.items())

per_release = len(s) / s["Publication"].nunique()
memo("§1, §5.1, §5.3", f"H1 sample: {len(s):,} observations over
{s['Publication'].nunique()} releases",
     ("103,920", f"{len(s):,}"))
memo("§4.1", "the same, rounded", ("about 103,900", f"about {round(len(s),
-2):,}"))
memo("§5.1", f"{s['DataRIC'].nunique()} firms (Table 2 note)", ("570",
str(s["DataRIC"].nunique())))
memo("§1, §4.1, §5.1", f"{per_release:.0f} stocks per release on average",
("433", f"{per_release:.0f}"))
table2_memo
```

```
[§1, §5.3, §5.1] H1 sample: 103,920 observations over 240 releases
[§4.1] the same, rounded
[§5.1] 570 firms (Table 2 note)
[§1, §4.1, §5.1] 433 stocks per release on average
```

Out[11]:

| | Mean | SD | P10 | Median | P90 |
|---|---|---|---|---|---|
| **Short interest / float (%)** | 2.95 | 2.72 | 0.93 | 2.04 | 6.09 |
| **5-day market-adjusted return (%)** | -0.08 | 3.67 | -4.20 | -0.07 | 4.04 |
| **Market cap ($ billion)** | 80.3 | 236.3 | 10.3 | 29.0 | 155.8 |
| **Book-to-market** | 0.37 | 0.31 | 0.07 | 0.29 | 0.76 |
| **Turnover (% of shares per day)** | 0.85 | 0.59 | 0.39 | 0.68 | 1.45 |
| **Zero-return days (% of last 21 days)** | 0.44 | 1.50 | 0.00 | 0.00 | 0.00 |
| **Idiosyncratic volatility (% per day)** | 1.46 | 0.66 | 0.83 | 1.30 | 2.29 |
| **Analysts (number)** | 18.3 | 7.8 | 9 | 18 | 28 |
| **Return, past month (%)** | 1.00 | 8.69 | -8.92 | 1.04 | 10.92 |
| **Return, months -6 to -2 (%)** | 4.93 | 19.61 | -17.52 | 3.96 | 27.56 |

In [12]:

```
sd_cs = s.groupby("Publication")["SI_float"].std().mean()          # 1 SD in
every coefficient
q50, q90 = s["SI_float"].quantile([0.5, 0.9])
corr = lambda a, b: s.groupby("Publication")[[a, b]].apply(lambda g:
g[a].corr(g[b])).mean()
c_mv, c_cov, c_covmv = corr("SI_float", "LogMV"), corr("SI_float",
"AnalystCov"), corr("AnalystCov", "LogMV")
memo("§1, §5.1", f"average cross-sectional SD of SI / float: {100 * sd_cs:.1f}
percentage points", ("2.6", f"{100 * sd_cs:.1f}"))
memo("§5.1", f"SI / float: mean {100 * s['SI_float'].mean():.2f}%, median {100 *
q50:.2f}%, 90th percentile {100 * q90:.2f}%",
     ("2.95%", f"{100 * s['SI_float'].mean():.2f}%"), ("2.04%", f"{100 *
q50:.2f}%"), ("6.09%", f"{100 * q90:.2f}%"))
memo("§5.1", f"five-day market-adjusted return: mean {100 * s[F.Y].mean():.2f}%,
SD {100 * s[F.Y].std():.2f}%",
     ("-0.08%", f"{100 * s[F.Y].mean():.2f}%"), ("3.67%", f"{100 *
```

```
s[F.Y].std():.2f}%"))
memo("§5.1", f"within-release correlations: SI-log size {c_mv:.2f}, SI-coverage
{c_cov:.2f}, coverage-log size {c_covmv:+.2f}",
     ("-0.46", f"{c_mv:.2f}"), ("-0.10", f"{c_cov:.2f}"), ("+0.46", f"
{c_covmv:+.2f}"))
```

```
[§1, §5.1] average cross-sectional SD of SI / float: 2.6 percentage points
[§5.1] SI / float: mean 2.95%, median 2.04%, 90th percentile 6.09%
[§5.1] five-day market-adjusted return: mean -0.08%, SD 3.67%
[§5.1] within-release correlations: SI-log size -0.46, SI-coverage -0.10,
coverage-log size +0.46
```

## Memo Table 3. Fama-MacBeth regressions of the next five-day market-adjusted return (memo §5.2, §5.4)

In [13]:

```
MAIN = {"(1)": "(1) SI only", "(2)": "(2) H1", "(3)": "(3) H2: coverage", "(4)":
"(4) H2: size",
        "(2) low coverage": "(2) low-coverage third", "(2) high coverage": "(2)
high-coverage third"}
TERMS = {"SI_float": "SI / float", "LogMV": "Log market cap", "BM": "Book-to-
market", "LagRet1m": "Return, past month",
         "LagRet6m": "Return, months -6 to -2", "Turnover": "Turnover",
"IdioVOL": "Idiosyncratic volatility",
         "PctZero": "% zero-return days", "AnalystCov": "Analyst coverage,
ln(1+N)", "SI_float:AnalystCov": "SI × analyst coverage",
         "SI_float:LogMV": "SI × size", "const": "Intercept"}
table3 = pd.DataFrame({label: [cell(m, t) for t in TERMS] for m, label in
MAIN.items()}, index=list(TERMS.values()))
for name, col, fmt in [("Releases", "dates", "{:.0f}"), ("Avg. stocks per
release", "avg_n", "{:.0f}"),
                       ("Avg. adjusted R²", "adjR2", "{:.3f}")]:
    table3.loc[name] = [fmt.format(C.loc[(m, "const"), col]) for m in MAIN]
MEMO_TABLE3 = {   # as printed in the memo, columns (1), (2), (3), (4), low- and
high-coverage third
    "SI / float": "-3.99 (-1.38) | -6.22*** (-2.89) | -5.04** (-2.36) | -0.89
(-0.34) | -8.83*** (-2.80) | -3.71 (-0.99)",
    "Log market cap": " | 3.48 (1.38) | 2.88 (1.15) | 6.75** (2.20) | 0.38
(0.16) | 6.43 (1.63)",
    "Book-to-market": " | -2.39 (-0.70) | -2.34 (-0.69) | -2.08 (-0.62) | -1.39
(-0.42) | -3.62 (-0.87)",
    "Return, past month": " | -3.30 (-0.80) | -3.24 (-0.78) | -3.91 (-0.97) |
-3.29 (-0.81) | -3.55 (-0.79)",
    "Return, months -6 to -2": " | -1.88 (-0.42) | -1.71 (-0.39) | -2.43 (-0.54)
| -0.96 (-0.21) | -1.46 (-0.26)",
    "Turnover": " | 4.13 (1.20) | 3.24 (0.95) | 4.11 (1.18) | 2.12 (0.52) | 4.22
(0.91)",
    "Idiosyncratic volatility": " | 1.87 (0.63) | 1.54 (0.53) | 1.75 (0.59) |
1.62 (0.46) | 6.06 (1.50)",
    "% zero-return days": " | 0.27 (0.27) | 0.40 (0.39) | 0.40 (0.39) | 1.29
(0.71) | 0.96 (0.49)",
    "Analyst coverage, ln(1+N)": " |  | 2.31 (1.01) |  |  | ",
    "SI × analyst coverage": " |  | 1.75 (1.22) |  |  | ",
    "SI × size": " |  |  | 5.95*** (2.94) |  | ",
    "Intercept": "-7.52 (-1.28) | -7.93 (-1.34) | -7.55 (-1.29) | -5.05 (-0.89)
| -11.51 (-1.58) | -3.11 (-0.62)",
    "Releases": "240 | 240 | 240 | 240 | 240 | 240",
    "Avg. stocks per release": "461 | 433 | 429 | 433 | 144 | 140",
    "Avg. adjusted R²": "0.018 | 0.135 | 0.142 | 0.137 | 0.135 | 0.152"}
CHECKS.extend(("Table 3", said, " | ".join(table3.loc[row])) for row, said in
```

```
MEMO_TABLE3.items())
table3
```

Out[13]:

| | (1) SI only | (2) H1 | (3) H2: coverage | (4) H2: size | (2) low-coverage third | (2) high-coverage third |
|---|---|---|---|---|---|---|
| **SI / float** | -3.99 (-1.38) | -6.22*** (-2.89) | -5.04** (-2.36) | -0.89 (-0.34) | -8.83*** (-2.80) | -3.71 (-0.99) |
| **Log market cap** | | 3.48 (1.38) | 2.88 (1.15) | 6.75** (2.20) | 0.38 (0.16) | 6.43 (1.63) |
| **Book-to-market** | | -2.39 (-0.70) | -2.34 (-0.69) | -2.08 (-0.62) | -1.39 (-0.42) | -3.62 (-0.87) |
| **Return, past month** | | -3.30 (-0.80) | -3.24 (-0.78) | -3.91 (-0.97) | -3.29 (-0.81) | -3.55 (-0.79) |
| **Return, months -6 to -2** | | -1.88 (-0.42) | -1.71 (-0.39) | -2.43 (-0.54) | -0.96 (-0.21) | -1.46 (-0.26) |
| **Turnover** | | 4.13 (1.20) | 3.24 (0.95) | 4.11 (1.18) | 2.12 (0.52) | 4.22 (0.91) |
| **Idiosyncratic volatility** | | 1.87 (0.63) | 1.54 (0.53) | 1.75 (0.59) | 1.62 (0.46) | 6.06 (1.50) |
| **% zero-return days** | | 0.27 (0.27) | 0.40 (0.39) | 0.40 (0.39) | 1.29 (0.71) | 0.96 (0.49) |
| **Analyst coverage, ln(1+N)** | | | 2.31 (1.01) | | | |
| **SI × analyst coverage** | | | 1.75 (1.22) | | | |
| **SI × size** | | | | 5.95*** (2.94) | | |
| **Intercept** | -7.52 (-1.28) | -7.93 (-1.34) | -7.55 (-1.29) | -5.05 (-0.89) | -11.51 (-1.58) | -3.11 (-0.62) |
| **Releases** | 240 | 240 | 240 | 240 | 240 | 240 |
| **Avg. stocks per release** | 461 | 433 | 429 | 433 | 144 | 140 |
| **Avg. adjusted $R^2$** | 0.018 | 0.135 | 0.142 | 0.137 | 0.135 | 0.152 |

**What this shows.** H1 predicted a positive short-interest coefficient. Model (2) gives a negative one that is significant at the 1% level, so H1 is not supported and the evidence points to informed short selling instead of covering pressure. Model (3) is H2's test: the interaction with coverage is positive and insignificant, and the low-coverage third shows a negative effect, so H2 fails both of its pre-registered conditions. Model (4) shows where the negative effect sits: the positive short interest × size term means it is concentrated in the smaller members.

## Memo Table 4. Standard errors for the short-interest coefficient in model (2) (memo §5.3)

Plain Fama-MacBeth and Newey-West with 4, 8 and 12 lags; then pooled OLS on the same within-release standardized regressors with returns demeaned by release (date fixed effects), with conventional, firm-clustered and date-clustered errors. The last row lets the seven control coefficients differ by release, as Fama-MacBeth does (Frisch-Waugh: residualize the return and short interest on each release's controls, then pool).

In [14]:
```
r2 = F.fm(p, F.Y, F.H1)
b = r2["series"]["SI_float"].to_numpy()
rows = [(f"Fama-MacBeth, {'no lag correction' if lags == 0 else f'Newey-West
{lags} lags'}",
         b.mean(), b.mean() / F.nw_tstat(b, lags), F.nw_tstat(b, lags)) for
lags in (0, 4, 8, 12)]

z = s.copy()
for c in F.H1:
    z[c] = z.groupby("Publication")[c].transform(lambda x: (x - x.mean()) /
x.std())
y = 1e4 * (z[F.Y] - z.groupby("Publication")[F.Y].transform("mean"))       #
date fixed effects
X, yy = z[F.H1].to_numpy(float), y.to_numpy(float)
XtX_inv = np.linalg.inv(X.T @ X)
beta = XtX_inv @ X.T @ yy
u = yy - X @ beta
n, k = X.shape


def clustered_se(groups):
    meat = np.zeros((k, k))
    for idx in
pd.Series(np.arange(n)).groupby(groups.to_numpy()).indices.values():
        g = X[idx].T @ u[idx]
        meat += np.outer(g, g)
    return np.sqrt(np.diag(XtX_inv @ meat @ XtX_inv))[0]


for name, se in [("conventional SE", np.sqrt(XtX_inv[0, 0] * (u @ u) / (n -
k))),
                 ("clustered by firm", clustered_se(z["DataRIC"])), ("clustered
by date", clustered_se(z["Publication"]))]:
    rows.append((f"Pooled OLS, date effects, {name}", beta[0], se, beta[0] /
se))

ry, rx, lab = [], [], []
for d, g in z.assign(y=y).groupby("Publication"):
    W = np.column_stack([np.ones(len(g)), g[F.CTRL].to_numpy(float)])
    for v, out in [("y", ry), ("SI_float", rx)]:
        out.append(g[v].to_numpy(float) - W @ np.linalg.lstsq(W,
g[v].to_numpy(float), rcond=None)[0])
    lab += [d] * len(g)
ry, rx = np.concatenate(ry), np.concatenate(rx)
b_fwl = rx @ ry / (rx @ rx)
se_fwl = np.sqrt((pd.Series(rx * (ry - b_fwl * rx)).groupby(lab).sum() **
2).sum()) / (rx @ rx)
```

```
rows.append(("Pooled OLS, control slopes vary by date, clustered by date",
b_fwl, se_fwl, b_fwl / se_fwl))

table4 = pd.DataFrame(rows, columns=["Method", "Coefficient", "Std. error", "t-
statistic"]).set_index("Method")
MEMO_TABLE4 = ["-6.22 2.70 -2.30", "-6.22 2.15 -2.89", "-6.22 2.06 -3.01",
"-6.22 2.05 -3.03",
               "-4.43 1.45 -3.06", "-4.43 1.92 -2.31", "-4.43 3.22 -1.38",
"-6.08 2.65 -2.30"]
CHECKS.extend(("Table 4", said, " ".join(f"{x:.2f}" for x in table4.iloc[i]))
for i, said in enumerate(MEMO_TABLE4))
rho = np.corrcoef(b[1:], b[:-1])[0, 1]
memo("§5.3", f"first-order autocorrelation of the {len(b)} release-by-release
slopes: {rho:.2f}", ("-0.16", f"{rho:.2f}"))
table4.round(2)
```

```
[§5.3] first-order autocorrelation of the 240 release-by-release slopes: -0.16
```

Out[14]:

| Method | Coefficient | Std. error | t-statistic |
|---|---|---|---|
| **Fama-MacBeth, no lag correction** | -6.22 | 2.70 | -2.30 |
| **Fama-MacBeth, Newey-West 4 lags** | -6.22 | 2.15 | -2.89 |
| **Fama-MacBeth, Newey-West 8 lags** | -6.22 | 2.06 | -3.01 |
| **Fama-MacBeth, Newey-West 12 lags** | -6.22 | 2.05 | -3.03 |
| **Pooled OLS, date effects, conventional SE** | -4.43 | 1.45 | -3.06 |
| **Pooled OLS, date effects, clustered by firm** | -4.43 | 1.92 | -2.31 |
| **Pooled OLS, date effects, clustered by date** | -4.43 | 3.22 | -1.38 |
| **Pooled OLS, control slopes vary by date, clustered by date** | -6.08 | 2.65 | -2.30 |

**What this shows.** The verdict on H1 depends on the model more than on the standard-error formula. Plain Fama-MacBeth, the Newey-West versions and the pooled regression that lets the controls vary by date all give significant negative t-statistics. Only the pooled regression that holds the control coefficients fixed for ten years does not. Because the release-by-release slopes are negatively autocorrelated, Newey-West makes the t-statistic larger, so the uncorrected Fama-MacBeth figure is the conservative one.

## Memo Table 5. Short-interest slope by year, model (2) (memo §5.5)

In [15]:

```
ser = r2["series"].set_index("date")["SI_float"]
yr = ser.groupby(ser.index.year)
table5 = pd.DataFrame({"Releases": yr.size(), "Mean slope (bps)": yr.mean(),
                       "t-statistic": [F.nw_tstat(x.to_numpy()) for _, x in yr],
                       "% of releases negative": yr.apply(lambda x: 100 * (x <
0).mean())})
table5.index = table5.index.astype(str)
for label, x in [("2016–2020", ser[:"2020"]), ("2021–2026", ser["2021":]), ("All
```

```
releases", ser)]:
    table5.loc[label] = [len(x), x.mean(), F.nw_tstat(x.to_numpy()), 100 * (x <
0).mean()]
MEMO_TABLE5 = {"2016": "7 -2.13 -0.27 57.1", "2017": "24 -13.82 -1.63 66.7",
"2018": "24 -3.97 -0.80 45.8",
               "2019": "24 -19.22 -4.10 75.0", "2020": "24 -8.02 -0.99 66.7",
"2021": "24 -2.73 -0.47 54.2",
               "2022": "24 -9.56 -1.85 58.3", "2023": "24 +5.26 0.82 50.0",
"2024": "24 -5.29 -0.94 62.5",
               "2025": "24 -7.27 -1.36 66.7", "2026": "17 +4.38 1.15 47.1",
"2016–2020": "103 -10.64 -3.05 63.1",
               "2021–2026": "137 -2.89 -1.16 56.9", "All releases": "240 -6.22
-2.89 59.6"}
CHECKS.extend(("Table 5", said, "{:.0f} {:+.2f} {:.2f}
{:.1f}".format(*table5.loc[row])) for row, said in MEMO_TABLE5.items())
table5.round(2)
```

Out[15]:

| date | Releases | Mean slope (bps) | t-statistic | % of releases negative |
|---|---|---|---|---|
| **2016** | 7.0 | -2.13 | -0.27 | 57.14 |
| **2017** | 24.0 | -13.82 | -1.63 | 66.67 |
| **2018** | 24.0 | -3.97 | -0.80 | 45.83 |
| **2019** | 24.0 | -19.22 | -4.10 | 75.00 |
| **2020** | 24.0 | -8.02 | -0.99 | 66.67 |
| **2021** | 24.0 | -2.73 | -0.47 | 54.17 |
| **2022** | 24.0 | -9.56 | -1.85 | 58.33 |
| **2023** | 24.0 | 5.26 | 0.82 | 50.00 |
| **2024** | 24.0 | -5.29 | -0.94 | 62.50 |
| **2025** | 24.0 | -7.27 | -1.36 | 66.67 |
| **2026** | 17.0 | 4.38 | 1.15 | 47.06 |
| **2016–2020** | 103.0 | -10.64 | -3.05 | 63.11 |
| **2021–2026** | 137.0 | -2.89 | -1.16 | 56.93 |
| **All releases** | 240.0 | -6.22 | -2.89 | 59.58 |

**What this shows.** Single releases are noisy: the slopes swing far more than their average, and only about six in ten are negative. Averaged by year the effect is negative in most years, strongest in 2019 and slightly positive in 2023 and 2026. It is clearly weaker after 2020, but the difference between the two halves of the sample is not statistically significant, so the memo treats the decline as a warning to monitor, not a proven break.

## Memo Table 6 (Appendix). Robustness: short-interest coefficient across specifications (memo §4.5, §5.2)

In [16]:

```
H2I = "SI_float:AnalystCov"
ROB = [   # (specification, model, SI term, other term, label of the other term)
    ("A. Baseline: 5-day return, equal-weighted", "(2)", "SI_float", None, ""),
    ("A. Value-weighted (WLS by market cap)", "(2) value-weighted", "SI_float",
None, ""),
    ("A. Short interest / shares outstanding", "(2) SI/shares", "SI_shares",
None, ""),
    ("A. Raw panel (no cleaning or winsorizing)", "(2) raw panel", "SI_float",
None, ""),
    ("A. Keep negative book equity (with dummy)", "(2) neg BE dummy",
"SI_float", None, ""),
    ("A. 5-day window starts one day later", "(2) skip1", "SI_float", None, ""),
    ("A. Bi-weekly return, release t to t+1", "(2) 2w t+1", "SI_float", None,
""),
    ("A. Bi-weekly return, release t+1 to t+2", "(2) 2w t+2", "SI_float", None,
""),
    ("A. Add ΔSI, 5-day return", "ΔSI 5d", "SI_float", "dSI_float", "ΔSI: "),
    ("A. Add ΔSI and SI × ΔSI, 5-day return", "SI×ΔSI 5d", "SI_float",
"SI_float:dSI_float", "SI × ΔSI: "),
    ("A. Add ΔSI, bi-weekly return", "ΔSI 2w", "SI_float", "dSI_float", "ΔSI:
"),
    ("A. Releases in 2016–2020", "(2) 2016-2020", "SI_float", None, ""),
    ("A. Releases in 2021–2026", "(2) 2021-2026", "SI_float", None, ""),
    ("B. Baseline", "(3)", "SI_float", H2I, ""),
    ("B. Value-weighted (WLS by market cap)", "(3) value-weighted", "SI_float",
H2I, ""),
    ("B. Bi-weekly return, release t to t+1", "(3) 2w t+1", "SI_float", H2I,
""),
    ("B. Releases in 2016–2020", "(3) 2016-2020", "SI_float", H2I, ""),
    ("B. Releases in 2021–2026", "(3) 2021-2026", "SI_float", H2I, ""),
]


def rob_cell(model, term):
    """Table 6 format: coefficient, t-statistic in brackets, then the stars."""
    b, t = coef(model, term)
    return f"{b:.2f} ({t:.2f}){F.stars(t)}"


table6 = pd.DataFrame([{"Specification": name, "SI / float": rob_cell(m, si),
"Other term": lab + rob_cell(m, o) if o else "",
                        "Releases": int(C.loc[(m, "const"), "dates"])} for name,
m, si, o, lab in ROB]).set_index("Specification")
MEMO_TABLE6 = [   # as printed in the memo: SI / float | other term | releases
    "-6.22 (-2.89)*** |  | 240", "-4.80 (-1.35) |  | 240", "-6.56 (-2.80)*** |
| 240", "-11.02 (-1.72)* |  | 240",
    "-5.94 (-2.82)*** |  | 240", "-4.32 (-1.70)* |  | 240", "-10.83 (-3.34)*** |
| 239", "-11.09 (-3.47)*** |  | 238",
    "-6.30 (-2.94)*** | ΔSI: -2.60 (-1.78)* | 239", "-6.77 (-3.12)*** | SI ×
ΔSI: 2.29 (1.37) | 239",
    "-11.17 (-3.43)*** | ΔSI: -3.10 (-1.53) | 238", "-10.64 (-3.05)*** |  |
103", "-2.89 (-1.16) |  | 137",
    "-5.04 (-2.36)** | 1.75 (1.22) | 240", "-3.27 (-0.95) | 3.79 (1.34) | 240",
"-9.48 (-2.96)*** | 1.22 (0.62) | 239",
    "-9.43 (-2.90)*** | 3.24 (1.54) | 103", "-1.75 (-0.66) | 0.63 (0.33) | 137"]
CHECKS.extend(("Table 6", said, " | ".join(map(str, table6.iloc[i]))) for i,
said in enumerate(MEMO_TABLE6))
print("Panel A: H1, model (2). Panel B: H2, model (3); other term = SI × analyst
```

```
coverage.")
table6
```

```
Panel A: H1, model (2). Panel B: H2, model (3); other term = SI × analyst
coverage.
```

Out[16]:

| Specification | SI / float | Other term | Releases |
|---|---|---|---|
| **A. Baseline: 5-day return, equal-weighted** | -6.22 (-2.89)*** | | 240 |
| **A. Value-weighted (WLS by market cap)** | -4.80 (-1.35) | | 240 |
| **A. Short interest / shares outstanding** | -6.56 (-2.80)*** | | 240 |
| **A. Raw panel (no cleaning or winsorizing)** | -11.02 (-1.72)* | | 240 |
| **A. Keep negative book equity (with dummy)** | -5.94 (-2.82)*** | | 240 |
| **A. 5-day window starts one day later** | -4.32 (-1.70)* | | 240 |
| **A. Bi-weekly return, release t to t+1** | -10.83 (-3.34)*** | | 239 |
| **A. Bi-weekly return, release t+1 to t+2** | -11.09 (-3.47)*** | | 238 |
| **A. Add ΔSI, 5-day return** | -6.30 (-2.94)*** | ΔSI: -2.60 (-1.78)* | 239 |
| **A. Add ΔSI and SI × ΔSI, 5-day return** | -6.77 (-3.12)*** | SI × ΔSI: 2.29 (1.37) | 239 |
| **A. Add ΔSI, bi-weekly return** | -11.17 (-3.43)*** | ΔSI: -3.10 (-1.53) | 238 |
| **A. Releases in 2016–2020** | -10.64 (-3.05)*** | | 103 |
| **A. Releases in 2021–2026** | -2.89 (-1.16) | | 137 |
| **B. Baseline** | -5.04 (-2.36)** | 1.75 (1.22) | 240 |
| **B. Value-weighted (WLS by market cap)** | -3.27 (-0.95) | 3.79 (1.34) | 240 |
| **B. Bi-weekly return, release t to t+1** | -9.48 (-2.96)*** | 1.22 (0.62) | 239 |
| **B. Releases in 2016–2020** | -9.43 (-2.90)*** | 3.24 (1.54) | 103 |
| **B. Releases in 2021–2026** | -1.75 (-0.66) | 0.63 (0.33) | 137 |

**What this shows.** The negative sign survives every change to the H1 model: scaling by shares outstanding, keeping negative book equity, the release-to-release horizons, and adding the change in short interest. It weakens in the two places that matter for the recommendation: value weighting, where it is not significant, and the 2021–2026 releases. The delayed window (one day later) is the closest to what a trader could capture, because exchanges release short interest after the close.

# 7. Numbers quoted in the memo text

In [17]:

```
b2, t2 = coef("(2)")
gap = (q90 - q50) / sd_cs
memo("§5.2", f"90th percentile of SI is {gap:.1f} SD above the median: {gap *
b2:.0f} bps over five days; "
             f"SD of five-day returns {100 * s[F.Y].std():.1f}%",
     ("6.1%", f"{100 * q90:.1f}%"), ("1.6", f"{gap:.1f}"), ("10 bps lower", f"
{-gap * b2:.0f} bps lower"),
     ("3.7%", f"{100 * s[F.Y].std():.1f}%"))

# why the controls make the SI coefficient more negative: one control at a time,
same H1 sample
alone = lambda terms: F.fm(s, F.Y, terms)["coefs"]["SI_float"][0]
b_si, b_turn, b_size = alone(["SI_float"]), alone(["SI_float", "Turnover"]),
alone(["SI_float", "LogMV"])
c_turn, c_vol = corr("SI_float", "Turnover"), corr("SI_float", "IdioVOL")
memo("§5.2", f"same sample: SI alone {b_si:.2f} bps, with turnover {b_turn:.2f},
with size {b_size:.2f}; within-release "
             f"correlation of SI with turnover {c_turn:+.2f}, with volatility
{c_vol:+.2f}",
     ("-4.38", f"{b_si:.2f}"), ("-6.37", f"{b_turn:.2f}"), ("-2.58", f"
{b_size:.2f}"), ("+0.58", f"{c_turn:+.2f}"),
     ("+0.43", f"{c_vol:+.2f}"))
sd_dsi = s.dropna(subset=["dSI_float"]).groupby("Publication")
["dSI_float"].std().mean()   # 1 SD of ΔSI
memo("§5.2", f"one standard deviation of the change in short interest: a {100 *
sd_dsi:.1f}% change between releases",
     ("about a 13% change", f"about a {100 * sd_dsi:.0f}% change"))

# model (4): the SI effect at k SD of size is b_SI + k * b_SIxSize on each date;
its t-statistic comes from that series
b4 = coef("(4)")[0]
r4 = F.fm(p, F.Y, F.H2_SIZE)["series"]
at = {k: (r4["SI_float"] + k * r4["SI_float:LogMV"]).to_numpy() for k in (-1,
1, 2)}
t_at = {k: F.nw_tstat(v) for k, v in at.items()}
big = (s.groupby("Publication")["LogMV"].transform(lambda v: (v - v.mean()) /
v.std()) > 2).mean()
memo("§5.4", f"SI effect at average size {b4:.2f} bps; one SD smaller
{at[-1].mean():.1f} (t = {t_at[-1]:.2f}); "
                       f"one SD larger {at[1].mean():+.1f} (t = {t_at[1]:.2f});
two SD larger {at[2].mean():+.1f} "
                       f"(t = {t_at[2]:.2f}), a group with {big:.0%} of
observations",
     ("-6.8", f"{at[-1].mean():.1f}"), ("-3.11", f"{t_at[-1]:.2f}"), ("+5.1",
f"{at[1].mean():+.1f}"),
     ("1.22", f"{t_at[1]:.2f}"), ("+11.0", f"{at[2].mean():+.1f}"), ("1.84", f"
{t_at[2]:.2f}"), ("about 5%", f"about {big:.0%}"))

ratio = ser.std() / abs(ser.mean())
neg_years = int((table5.loc[[str(y) for y in range(2016, 2027)], "Mean slope
(bps)"] < 0).sum())
memo("§5.5", f"SD of the {len(ser)} slopes {ser.std():.1f} bps, {ratio:.2f}x
the mean; {100 * (ser < 0).mean():.1f}% negative; "
             f"mean negative in {neg_years} of 11 years",
     ("42", f"{ser.std():.0f}"), ("nearly seven times", "nearly seven times" if
6.5 <= ratio < 7 else f"{ratio:.1f} times"),
     ("59.6%", f"{100 * (ser < 0).mean():.1f}%"), ("9 of the 11", f"{neg_years}
of the 11"))
memo("Table 5 note", f"SD of the slopes to one decimal: {ser.std():.1f} bps",
```

```
("42.0", f"{ser.std():.1f}"))

roll = ser.rolling(24).mean()
x = ser.to_numpy()
h = len(x) // 2
se_mean = lambda v: v.mean() / F.nw_tstat(v)
diff = x[:h].mean() - x[h:].mean()
t_diff = diff / np.hypot(se_mean(x[:h]), se_mean(x[h:]))
memo("§5.5", f"rolling one-year (24-release) mean from {roll.min():.1f} to
{roll.max():+.1f} bps; two halves of {h} releases differ "
             f"by {diff:.2f} bps (t = {t_diff:.2f})",
     ("-21.7", f"{roll.min():.1f}"), ("+6.1", f"{roll.max():+.1f}"), ("-6.89",
f"{diff:.2f}"), ("-1.64", f"{t_diff:.2f}"))

skip = abs(coef("(2) skip1")[0])   # data come out after the close: the tradable
effect lies between skip1 and model (2)
memo("§1", f"effect a trader could capture: {skip:.2f} to {abs(b2):.2f} bps per
SD over five days",
     ("4 to 6 bps", f"{skip:.0f} to {abs(b2):.0f} bps"))
b2w = abs(coef("(2) 2w t+1")[0])
memo("§1", f"the headline in words: {abs(b2):.1f} basis points over five days,
about {b2w:.0f} bps release to release",
     ("6.2", f"{abs(b2):.1f}"), ("about 11 bps", f"about {b2w:.0f} bps"))

missing_cov = 1 - len(s.dropna(subset=["AnalystCov"])) / len(s)
memo("§6.2", f"analyst count missing for {missing_cov:.1%} of H1-sample rows",
     ("about 1%", "about 1%" if 0.005 <= missing_cov < 0.015 else f"
{missing_cov:.1%}"))
memo("§4.1", f"H2 regressions: {C.loc[('(3)', 'const'), 'avg_n']:.0f} stocks per
release",
     ("429", f"{C.loc[('(3)', 'const'), 'avg_n']:.0f}"))
```

```
[§5.2] 90th percentile of SI is 1.6 SD above the median: -10 bps over five days;
SD of five-day returns 3.7%
[§5.2] same sample: SI alone -4.38 bps, with turnover -6.37, with size -2.58;
within-release correlation of SI with turnover +0.58, with volatility +0.43
[§5.2] one standard deviation of the change in short interest: a 12.9% change
between releases
[§5.4] SI effect at average size -0.89 bps; one SD smaller -6.8 (t = -3.11); one
SD larger +5.1 (t = 1.22); two SD larger +11.0 (t = 1.84), a group with 5% of
observations
[§5.5] SD of the 240 slopes 42.0 bps, 6.75x the mean; 59.6% negative; mean
negative in 9 of 11 years
[Table 5 note] SD of the slopes to one decimal: 42.0 bps
[§5.5] rolling one-year (24-release) mean from -21.7 to +6.1 bps; two halves of
120 releases differ by -6.89 bps (t = -1.64)
[§1] effect a trader could capture: 4.32 to 6.22 bps per SD over five days
[§1] the headline in words: 6.2 basis points over five days, about 11 bps release
to release
[§6.2] analyst count missing for 0.8% of H1-sample rows
[§4.1] H2 regressions: 429 stocks per release
```

In [18]:
```
# Executive summary (§1) and §5.2-§5.4: the headline estimates, as the memo
quotes them
for where, label, model, term, said_b, said_t in [
        ("§5.2", "H1 model (2), SI / float", "(2)", "SI_float", "-6.22",
"-2.89"),
        ("§5.2", "SI only, model (1)", "(1)", "SI_float", "-3.99", "-1.38"),
        ("§5.2", "bi-weekly return, release t to t+1", "(2) 2w t+1", "SI_float",
"-10.83", "-3.34"),
```

```
        ("§5.2", "bi-weekly return, release t+1 to t+2", "(2) 2w t+2",
"SI_float", "-11.09", "-3.47"),
        ("§5.2", "value-weighted", "(2) value-weighted", "SI_float", "-4.80",
"-1.35"),
        ("§5.2", "5-day window one day later", "(2) skip1", "SI_float", "-4.32",
"-1.70"),
        ("§5.2", "ΔSI, 5-day return", "ΔSI 5d", "dSI_float", "-2.60", "-1.78"),
        ("§5.2", "SI next to ΔSI", "ΔSI 5d", "SI_float", "-6.30", "-2.94"),
        ("§5.2", "SI × ΔSI", "SI×ΔSI 5d", "SI_float:dSI_float", "2.29", "1.37"),
        ("§1, §5.4", "SI × analyst coverage, model (3)", "(3)",
"SI_float:AnalystCov", "1.75", "1.22"),
        ("Table 3", "SI at average coverage, model (3)", "(3)", "SI_float",
"-5.04", "-2.36"),
        ("§1, §5.4", "low-coverage third", "(2) low coverage", "SI_float",
"-8.83", "-2.80"),
        ("§5.4", "high-coverage third", "(2) high coverage", "SI_float",
"-3.71", "-0.99"),
        ("§1, §5.4", "SI × size, model (4)", "(4)", "SI_float:LogMV", "5.95",
"2.94"),
        ("Table 6", "SI × coverage, value-weighted", "(3) value-weighted",
"SI_float:AnalystCov", "3.79", "1.34"),
        ("Table 6", "SI × coverage, bi-weekly", "(3) 2w t+1",
"SI_float:AnalystCov", "1.22", "0.62"),
        ("§5.5", "2016–2020", "(2) 2016-2020", "SI_float", "-10.64", "-3.05"),
        ("§5.5", "2021–2026", "(2) 2021-2026", "SI_float", "-2.89", "-1.16"),
        ("§5.2", "SI / shares outstanding", "(2) SI/shares", "SI_shares",
"-6.56", "-2.80"),
        ("§5.2", "keeping negative book equity (dummy)", "(2) neg BE dummy",
"SI_float", "-5.94", "-2.82")]:
    bb, tt = coef(model, term)
    memo(where, f"{label}: {bb:.2f} bps (t = {tt:.2f})", (said_b, f"{bb:.2f}"),
(said_t, f"{tt:.2f}"))
# what the null results rule out: 95% confidence intervals, coefficient ± 1.96
Newey-West standard errors
for where, label, model, term, said_lo, said_hi in [
        ("§1, §5.2", "H1, model (2)", "(2)", "SI_float", "-10.4", "-2.0"),
        ("§5.4", "SI × coverage, model (3)", "(3)", "SI_float:AnalystCov",
"-1.1", "+4.6"),
        ("§5.4", "low-coverage third", "(2) low coverage", "SI_float", "-15.0",
"-2.7"),
        ("§5.5", "2021–2026", "(2) 2021-2026", "SI_float", "-7.8", "+2.0")]:
    bb, tt = coef(model, term)
    lo, hi = bb - 1.96 * bb / tt, bb + 1.96 * bb / tt
    memo(where, f"95% confidence interval, {label}: {lo:.1f} to {hi:+.1f} bps",
         (said_lo, f"{lo:.1f}"), (said_hi, f"{hi:+.1f}" if
said_hi.startswith("+") else f"{hi:.1f}"))
memo("§5.3, §6.2", f"pooled OLS with fixed control coefficients, clustered by
date: t = {table4.iloc[6, 2]:.2f}",
     ("-1.38", f"{table4.iloc[6, 2]:.2f}"))

# §6.2: delisted firms without short interest against the firms in the
regressions (Table 7 of regression_results.md)
import re
md_text = Path("regression_results.md").read_text(encoding="utf-8")
kept, gone, t_gap = re.search(r"\| next 5-day adj\. return \(bps\) \| (-?[\d.]+)
\| (-?[\d.]+) \| -?[\d.]+ \((-?[\d.]+)\)",
                              md_text).groups()
memo("§6.2", f"five-day return, firms in the regressions {float(kept):.1f} bps
against delisted firms without SI "
```

```
            f"{float(gone):.1f} bps (t = {float(t_gap):.2f})",
    ("-7.5", f"{float(kept):.1f}"), ("-11.5", f"{float(gone):.1f}"), ("-0.62",
f"{float(t_gap):.2f}"))
```

```
[§5.2] H1 model (2), SI / float: -6.22 bps (t = -2.89)
[§5.2] SI only, model (1): -3.99 bps (t = -1.38)
[§5.2] bi-weekly return, release t to t+1: -10.83 bps (t = -3.34)
[§5.2] bi-weekly return, release t+1 to t+2: -11.09 bps (t = -3.47)
[§5.2] value-weighted: -4.80 bps (t = -1.35)
[§5.2] 5-day window one day later: -4.32 bps (t = -1.70)
[§5.2] ΔSI, 5-day return: -2.60 bps (t = -1.78)
[§5.2] SI next to ΔSI: -6.30 bps (t = -2.94)
[§5.2] SI × ΔSI: 2.29 bps (t = 1.37)
[§1, §5.4] SI × analyst coverage, model (3): 1.75 bps (t = 1.22)
[Table 3] SI at average coverage, model (3): -5.04 bps (t = -2.36)
[§1, §5.4] low-coverage third: -8.83 bps (t = -2.80)
[§5.4] high-coverage third: -3.71 bps (t = -0.99)
[§1, §5.4] SI × size, model (4): 5.95 bps (t = 2.94)
[Table 6] SI × coverage, value-weighted: 3.79 bps (t = 1.34)
[Table 6] SI × coverage, bi-weekly: 1.22 bps (t = 0.62)
[§5.5] 2016–2020: -10.64 bps (t = -3.05)
[§5.5] 2021–2026: -2.89 bps (t = -1.16)
[§5.2] SI / shares outstanding: -6.56 bps (t = -2.80)
[§5.2] keeping negative book equity (dummy): -5.94 bps (t = -2.82)
[§1, §5.2] 95% confidence interval, H1, model (2): -10.4 to -2.0 bps
[§5.4] 95% confidence interval, SI × coverage, model (3): -1.1 to +4.6 bps
[§5.4] 95% confidence interval, low-coverage third: -15.0 to -2.7 bps
[§5.5] 95% confidence interval, 2021–2026: -7.8 to +2.0 bps
[§5.3, §6.2] pooled OLS with fixed control coefficients, clustered by date: t =
-1.38
[§6.2] five-day return, firms in the regressions -7.5 bps against delisted firms
without SI -11.5 bps (t = -0.62)
```

# 8. Check: every figure above matches the memo

In [19]:
```
check = pd.DataFrame(CHECKS, columns=["memo section", "memo says", "notebook"])
check["match"] = check["memo says"] == check["notebook"]
print(f"{check['match'].sum()} of {len(check)} memo figures match the values
computed in this notebook")
assert check["match"].all(), check[~check["match"]]
```

```
186 of 186 memo figures match the values computed in this notebook
```